Software Testing

Minsoo Ryu

Hanyang University

Topics covered

1. Testing Goals and Principles
2. Testing Levels
   - Component testing
   - Integration testing
   - System testing
   - Acceptance testing

Terminology

- An error is a mistake made by an engineer
  - often a misunderstanding of a requirement or design specification
- A fault is a manifestation of that error in the code
  - what we often call “a bug” (or “a defect”)
- A failure is an incorrect output/behavior that is caused by executing a fault
  - The failure may occur immediately (crash!) or much, much later in the execution
- Testing attempts to surface failures in our software systems
  - Debugging attempts to associate failures with faults so they can be removed from the system
- If a system passes all of its tests, is it free of all faults?

Goals of Testing

- Testing goals (Glen Meyer)
  - Testing is a process of executing a program with the intent of finding an error
  - A successful test is one that uncovers an as-yet-undiscovered error (fault)

- "Testing shows the presence, not the absence of bugs" —Edsger W. Dijkstra

- A good test case is one that has a high probability of finding an as-yet-undiscovered error
  - A test case is a particular choice of input data to be used in testing a program and the expected output or behavior

Some Principles about Testing

- All tests should be traceable to customer requirements
- Tests should be planned long before testing begins
  - Test planning can begin as soon as the analysis model complete
- The Pareto principle applies to software testing
  - 80 percent of all errors uncovered during testing will likely be traceable to 20 percent of all program components
  - The Pareto principle (also known as the 80-20 rule, the law of the vital few and the principle of factor sparsity) states that for many phenomena, 80% of the consequences stem from 20% of the causes
- Testing should begin "in the small" and progress toward testing "in the large"
- Exhaustive testing is not possible

Topics covered

1. Testing Goals and Principles
2. Testing Levels
   - Component testing
   - Integration testing
   - System testing
   - Acceptance testing

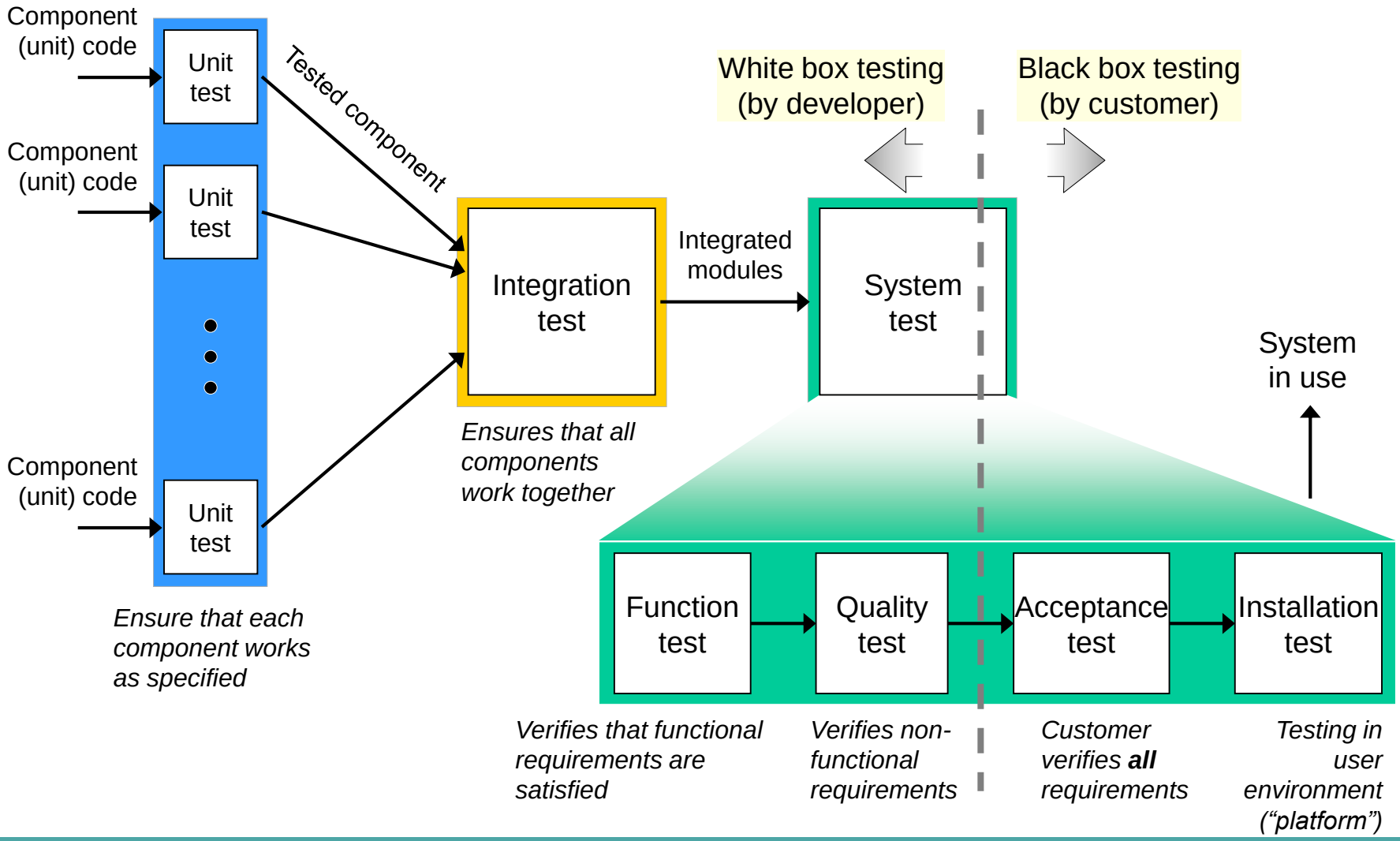
Testing Levels

- Testing begins “in the small” and progresses “to the large”

- Four levels
  - Unit testing
  - Integration testing
  - System testing
  - Acceptance testing

Level	Definition and Purpose
Unit Testing	Individual units/components of a software/system are tested. The purpose is to validate that each unit of the software performs as designed.
Integration Testing	Individual units are combined and tested as a whole. The purpose is to expose faults in the interaction between integrated units.
System Testing	A complete, integrated system/software is tested. The purpose is to evaluate the system’s compliance with the specified requirements.
Acceptance Testing	A system is tested for acceptability. The purpose is to evaluate the system’s compliance with the business requirements and assess whether it is acceptable for delivery.

Testing Levels



Unit Testing

- Individual units/components of a software/system are tested is to validate that each unit of the software performs as designed

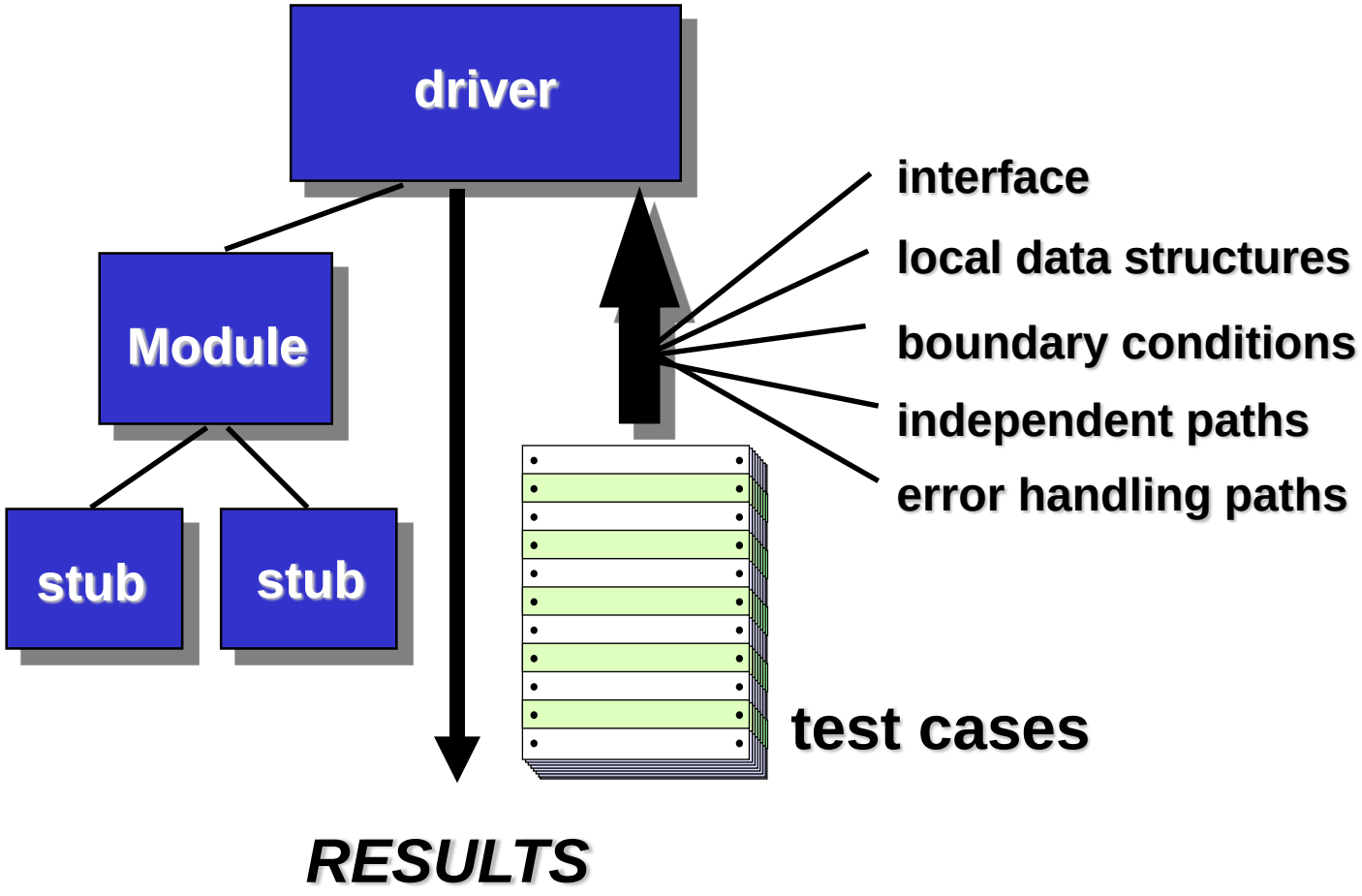
- Each module is tested to uncover errors within the boundary of the module
  - For each module, does each operation perform as expected
  - For method foo(), we'd like to see another method testFoo()

Unit Testing

- **Unit test environment**
  - **A driver is a "main program" that accepts tests case data, passes such data to the component, and prints relevant results**
  - **Stubs serve to replace modules that are subordinate to the component to be tested**
  - **A stub or "dummy subprogram" uses the subordinate module' interface, may do minimal data manipulation, provides verification of entry, and returns control to the module undergoing testing**
    - Mock or fake objects for simulation

- **Some comments on unit testing**
  - **Drivers and stubs are overhead (not delivered)**
  - **Unit testing is simplified when a component with high cohesion is designed**

Unit Test Environment



xUnit and JUnit

- Verification of the expected result is usually done using the assert_*_() methods that define the expected state and report errors if the actual state differs

- http://www.junit.org/

- Examples:
  - assertTrue(4 == (2 * 2));
  - assertEquals(expected, actual);
  - assertNull(Object object);

JUnit

```
import org.junit.*;
import static org.junit.Assert.*;
import java.util.*;

public class JunitTest1 {

    private Collection collection;

    @BeforeClass
    public static void oneTimeSetUp() {
        // one-time initialization code
      System.out.println("@BeforeClass - oneTimeSetUp");
    }

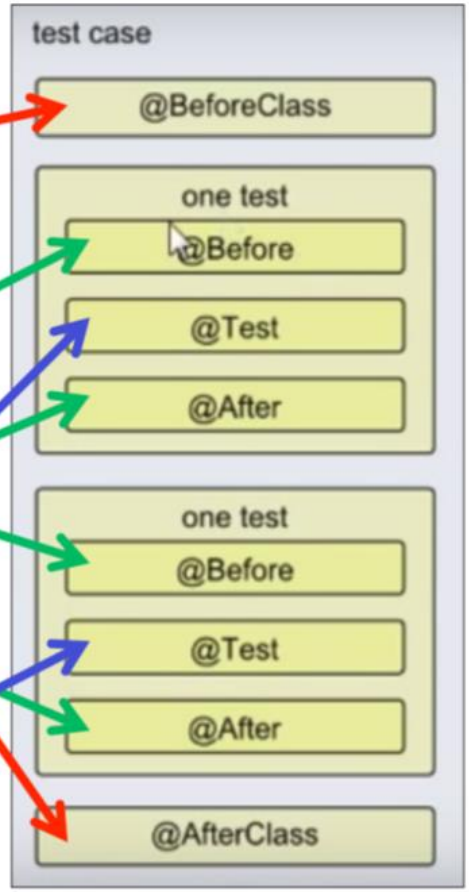
    @AfterClass
    public static void oneTimeTearDown() {
        // one-time cleanup code
      System.out.println("@AfterClass - oneTimeTearDown");
    }

    @Before
    public void setUp() {
        collection = new ArrayList();
        System.out.println("@Before - setUp");
    }

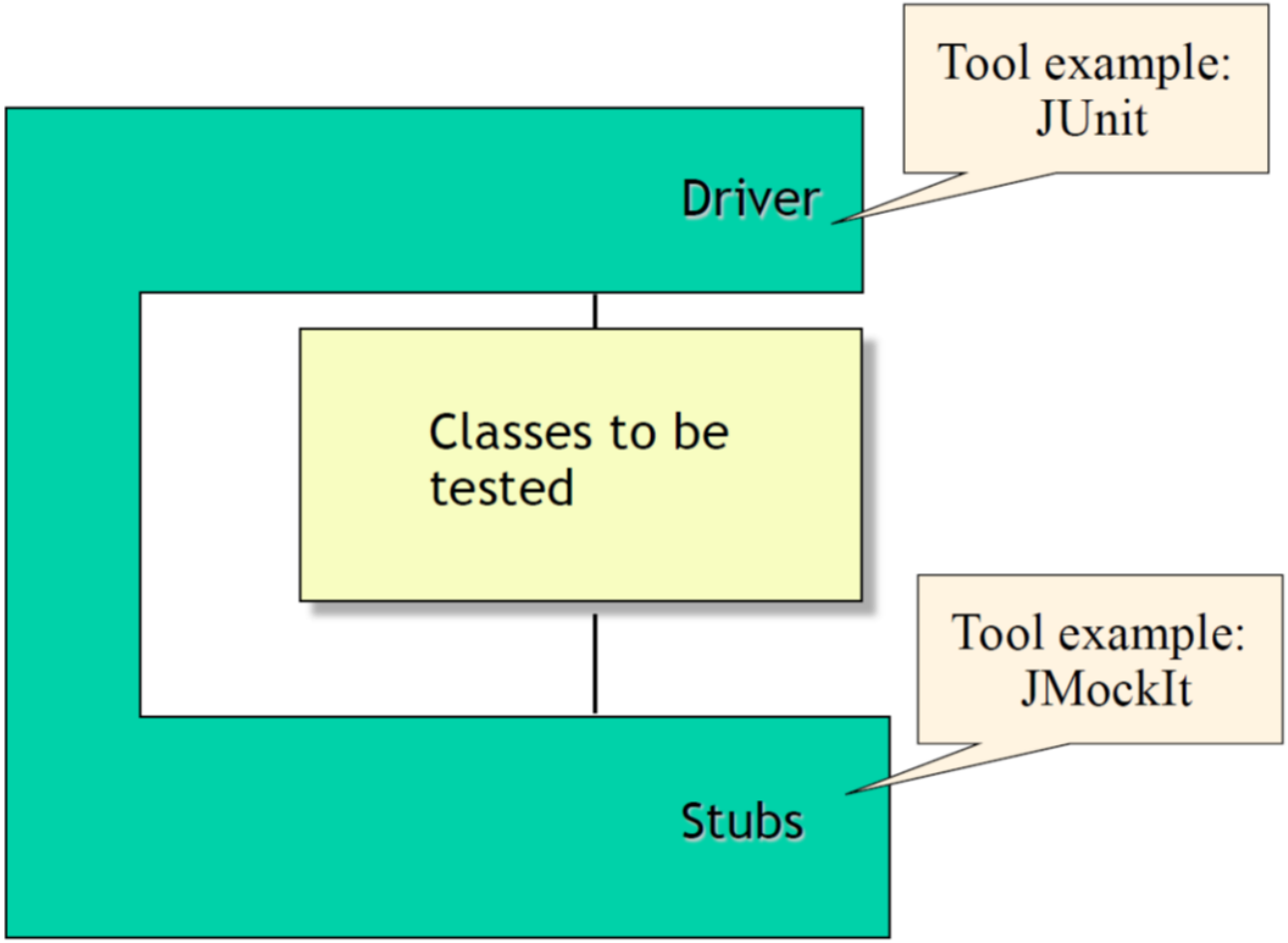
    @After
    public void tearDown() {
        collection.clear();
        System.out.println("@After - tearDown");
    }

    @Test
    public void testEmptyCollection() {
        assertTrue(collection.isEmpty());
        System.out.println("@Test - testEmptyCollection");
    }

    @Test
    public void testOneItemCollection() {
        collection.add("itemA");
        assertEquals(1, collection.size());
        System.out.println("@Test - testOneItemCollection");
    }
}
```

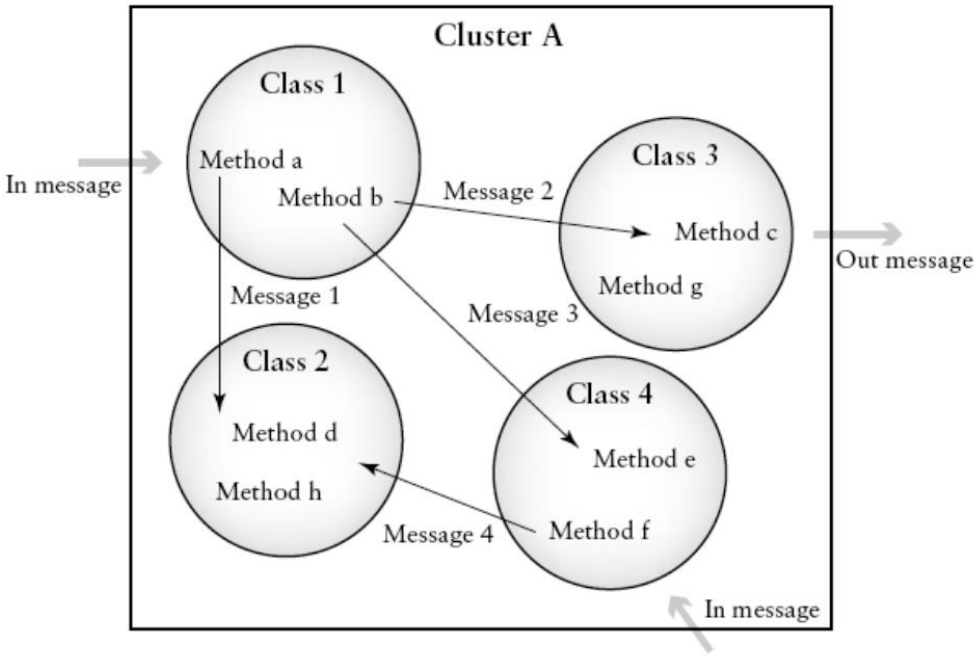


JUnit and JMockIt

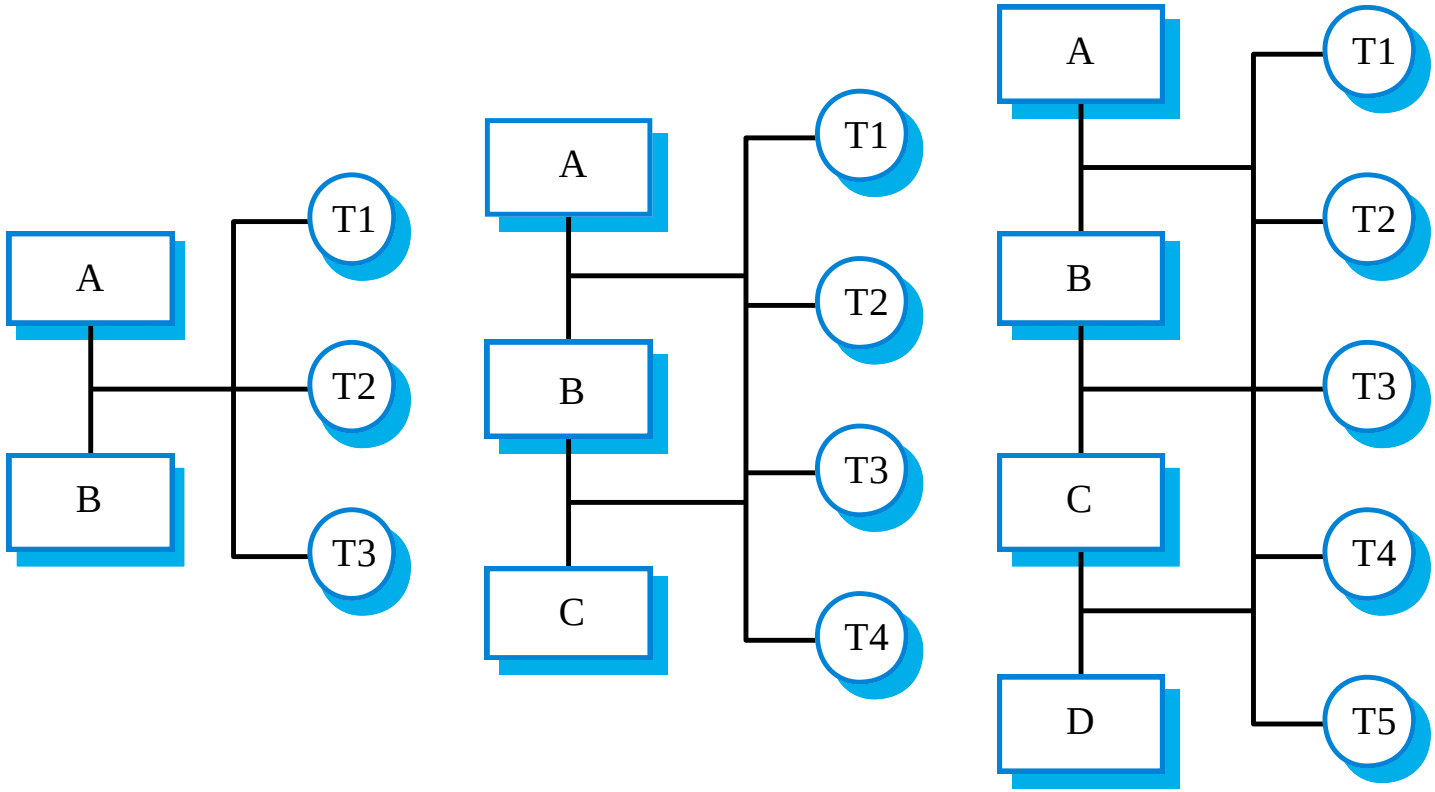


Integration Testing

- Individual units are combined and tested as a whole
- The purpose is to expose faults in the interaction between integrated units



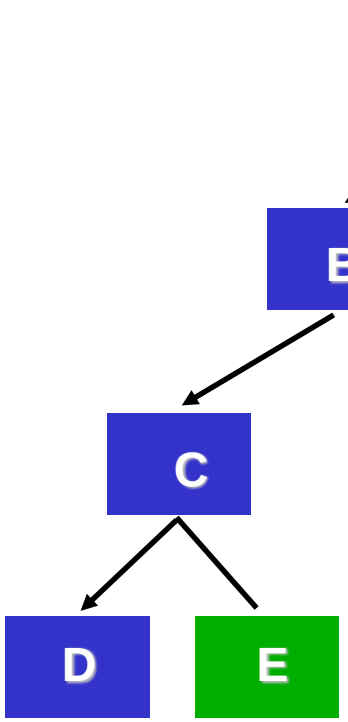
Incremental Integration Testing



Top-Down Integration

- Modules are integrated by moving downward through the control hierarchy, beginning with the main control module
  - Modules subordinate to the main control module are incorporated into the structure in either a depth-first or breadth-first manner
  - The top-down integration strategy verifies major control or decision points early in the test process
  - Stubs replace low-level modules at the beginning of top-down testing; therefore no significant data can flow upward in the program structure

Top-Down Integration



top module is tested with stubs

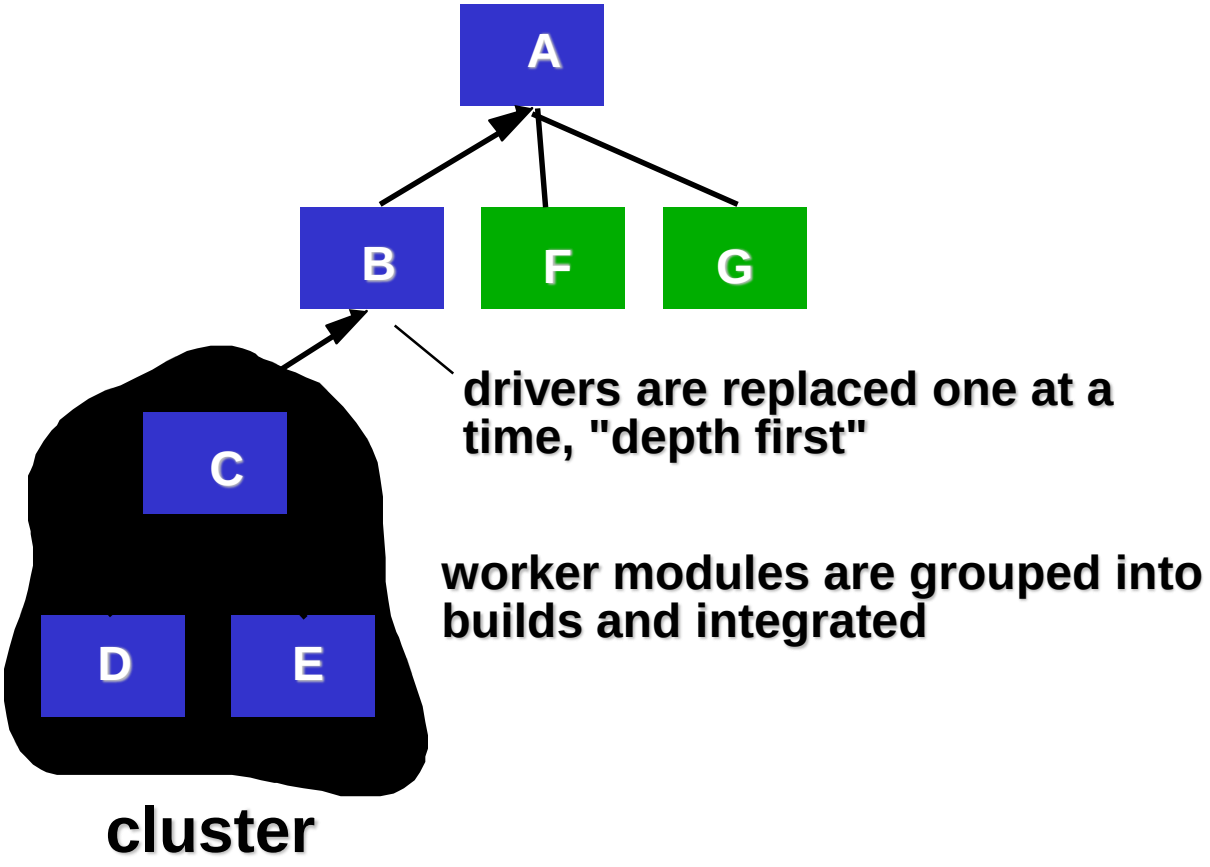
stubs are replaced one at a time, "depth first"

as new modules are integrated, some subset of tests is re-run

Bottom-Up Integration

- Bottom-up integration testing begins construction and testing with atomic modules
  - Low-level components are combined into clusters (builds)
  - A driver is written to coordinate test case input and output
  - The cluster is tested
  - Derivers are removed and clusters are combined moving upward in the program structure

Bottom-Up Integration



Regression Testing

- Each time a new module is added as part of integration testing, the software changes
  - New data flow paths are established, new I/O may occur, and new control logic is invoked

- Regression testing is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects

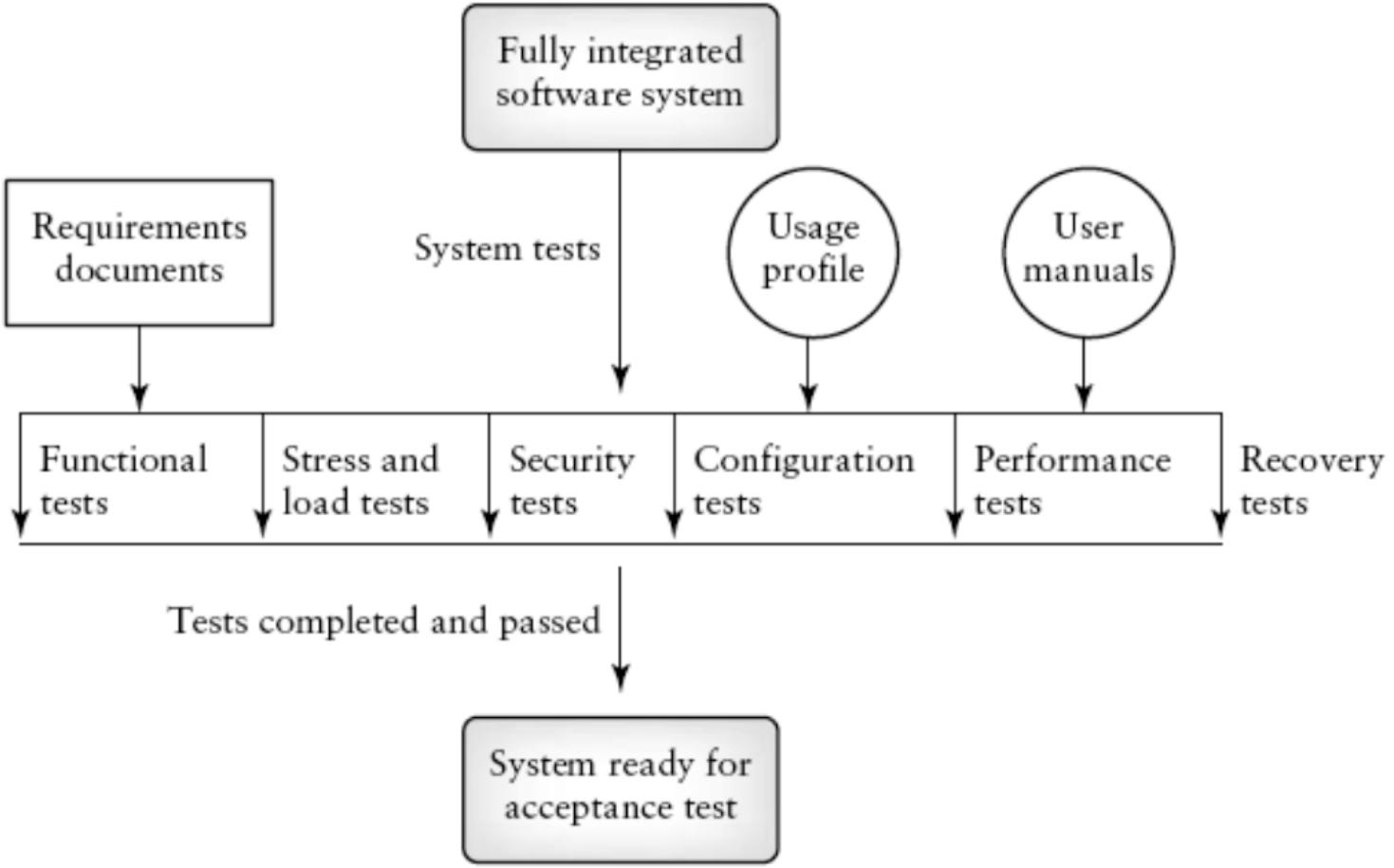
System Testing

- A complete, integrated system/software is tested

- The purpose is to evaluate the system's compliance with the specified requirements
  - Functional requirements
  - Non-functional requirements
    - Performance, stress, configuration, security, ...
    - As important as functional requirements
    - Often poorly specified, but must be tested

- Often done by independent test group
  - Collaborating with developers

System Testing



Acceptance Testing

- A system is tested for acceptability
  - The purpose is to evaluate the system's compliance with the business requirements and assess whether it is acceptable for delivery
- Internal Acceptance Testing
  - Performed by members of the organization that developed the software but who are not directly involved in the project (Development or Testing)
  - Usually, it is the members of Product Management, Sales and/or Customer Support
- External Acceptance Testing
  - Performed by people who are not employees of the organization that developed the software
  - Customer Acceptance Testing is performed by the customers of the organization that developed the software
    - They are the ones who asked the organization to develop the software
  - User Acceptance Testing is performed by the end users of the software
    - They can be the customers themselves or the customers' customers

Use Case Testing

- Use cases are very useful for designing acceptance tests with customer/user participation

- Acceptance tests can be derived from use cases
  - A use case describes interactions between actors, including users and the system, which produce a result of value to a system user or the customer
  - They also help uncover integration defects caused by the interaction and interference of different components, which individual component testing would not see

Alpha and Beta Testing

❒ Alpha testing

- Alpha test is conducted in a controlled manner at the developer's site by end-users
- The software is used in a natural setting with the developer looking over the shoulder of typical users and recording errors and usage problems

❒ Beta testing

- Beta test is conducted at end-user sites
- The developer is generally not present
- The beta test is a live application of the software in an environment that cannot be controlled by the developer
- The end-user records and reports all problems


Q & A